

---

**PROJECT TITLE: Student performance Analysis**

**GROUP MEMBERS: Yaseen khan, Hoorain eman, Amin ul haq**

**SECTION: "C"**

**DEPARTMENT OF COMPUTER SCEINCE**

**UNIVERSITY OF ENGINEERING & TECHNOLOGY,  
PESHAWAR**

---

# PROJECT REPORT

**SUBJECT: INTRODUCTION TO DATA SCIENCE**

**INSTRUCTOR: RIDA ZARKASH**

## Table of Contents

|                                      |    |
|--------------------------------------|----|
| Abstract.....                        | 2  |
| Introduction .....                   | 3  |
| Problem Statement.....               | 3  |
| Dataset Description.....             | 3  |
| Data Preprocessing .....             | 4  |
| Exploratory Data Analysis .....      | 4  |
| Feature Engineering.....             | 8  |
| Feature Selection .....              | 8  |
| Model Development .....              | 10 |
| Results and Discussion .....         | 10 |
| Conclusions and Recommendations..... | 13 |
| References .....                     | 13 |
| GitHub Repository.....               | 14 |

## Abstract

This project presents an end-to-end machine learning pipeline for predicting student academic performance before final exam results are available. Using the UCI Student Performance dataset (1,044 records, 34 attributes) covering Mathematics and Portuguese courses, we built a binary classifier to flag students likely to fail (final grade G3 below 10) using only demographic, social, and school-related attributes — deliberately excluding the earlier grade columns (G1, G2) to preserve genuine early-warning value. After data cleaning, engineering six new features, and selecting 16 final predictors through a four-method majority vote, we trained and compared Logistic Regression, Random Forest, Gradient Boosting, and SVM models. Gradient Boosting achieved the best ROC AUC (0.75), while every model reached 79–80% accuracy. Prior failures, absences, and parental education emerged as the strongest predictors. The results show that reliable early-risk flagging is achievable using only enrollment-time data, supporting timely academic intervention.

# Introduction

Schools typically identify struggling students only after final exam results are in, by which point interventions such as tutoring or counselling have limited impact. This project explores whether at-risk students can be flagged early, using only information available at enrollment, before a single test score exists. We approached this as a complete data science pipeline: acquiring and cleaning the data, exploring it, engineering new features, training and tuning several classification models, and comparing their performance. The goal was a working early-warning classifier rather than a single accuracy figure. Using the UCI Student Performance dataset, we framed the task as binary classification — predicting whether a student's final grade (G3) meets a passing threshold — based on demographic, social, and school-related attributes. Prior grade columns were intentionally excluded to keep the prediction genuinely early rather than trivially derived from scores the school already has.

## Problem Statement

The core problem is simple to state but harder to solve: schools don't have a systematic, data-driven way of spotting which students are likely to fail until it has already happened. If teachers and administrators could get a reasonably reliable signal at the start of the year, they could direct support — tutoring, counselling, parental engagement — toward the students who need it most, hopefully improving pass rates and cutting down on dropouts.

We framed this as a binary classification task: predict whether a student passes (final grade  $G3 \geq 10$ ) using only demographic, social, and school-related attributes. The two earlier grade columns, G1 and G2, are deliberately left out of the feature set. Including them would make the problem trivial, since they are almost the same score just measured earlier, and it would defeat the entire purpose of an early prediction system.

## Dataset Description

We used the well-known Student Performance dataset from the UCI Machine Learning Repository (Cortez & Silva, 2008), which is publicly available. It comes as two separate files — one for a Mathematics course and one for Portuguese — which we combined into a single dataset of 1,044 records across 34 columns (the 33 original attributes plus a subject flag marking which course each record came from). That comfortably clears the minimum bar of 1,000 records and 10+ features.

The attributes fall into a few natural groups:

Demographics — age, sex, home address type, family size

Parental background — mother's and father's education level, their jobs

School-related factors — weekly study time, number of past class failures, extra school support, absences

Social and lifestyle factors — how often the student goes out, weekday/weekend alcohol consumption, whether they're in a relationship, free time

Grades — G1, G2, and G3, each on a 0–20 scale

The target variable, passed, is derived from  $G3 \geq 10$ . The classes aren't perfectly balanced — roughly 78% of students pass and 22% fail — which turned out to matter later when we looked at recall on the fail class. A complete data dictionary for every column is included alongside the notebook.

# Data Preprocessing

Before jumping into modelling, we ran a basic data-quality pass to see what we were actually working with:

| Issue Checked                   | Finding                                                                  | Action Taken                                               |
|---------------------------------|--------------------------------------------------------------------------|------------------------------------------------------------|
| Missing values                  | None found                                                               | No imputation was needed                                   |
| Duplicate rows                  | 0 exact duplicates after merging the two files                           | Verified with <code>drop_duplicates()</code>               |
| Outliers (IQR rule)             | Mainly in absences (max value of 93, strongly right-skewed) and failures | Absences capped at the 99th percentile and log-transformed |
| Data types                      | Grade columns were read in as strings in one of the two files            | Forced to numeric with <code>pd.to_numeric</code>          |
| Inconsistent categorical values | None — categorical levels matched the official documentation             | No action required                                         |

Beyond the cleanup, we one-hot encoded the nominal categorical columns (Mjob, Fjob, reason, guardian) and label-encoded 14 binary attributes. All numeric features were scaled using `StandardScaler`, and — importantly — the scaler was fit only on the training split to avoid leaking any test-set information. The data was split 80/20 (835 training records, 209 test records) using stratified sampling to preserve the pass/fail ratio in both sets.

## Exploratory Data Analysis

We produced a fairly standard set of EDA visuals — histograms, boxplots, a correlation heatmap, scatter plots, count plots, and a pair plot — all saved under `outputs/figures/` for reference.

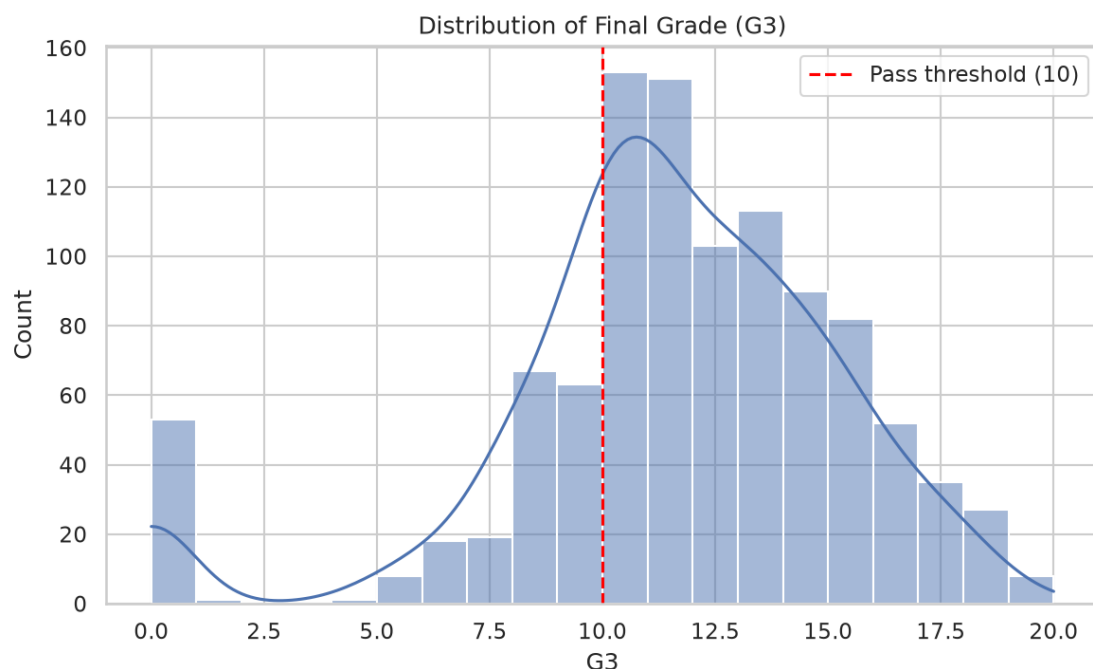


Figure 5.1: Distribution of Final Grade (G3)

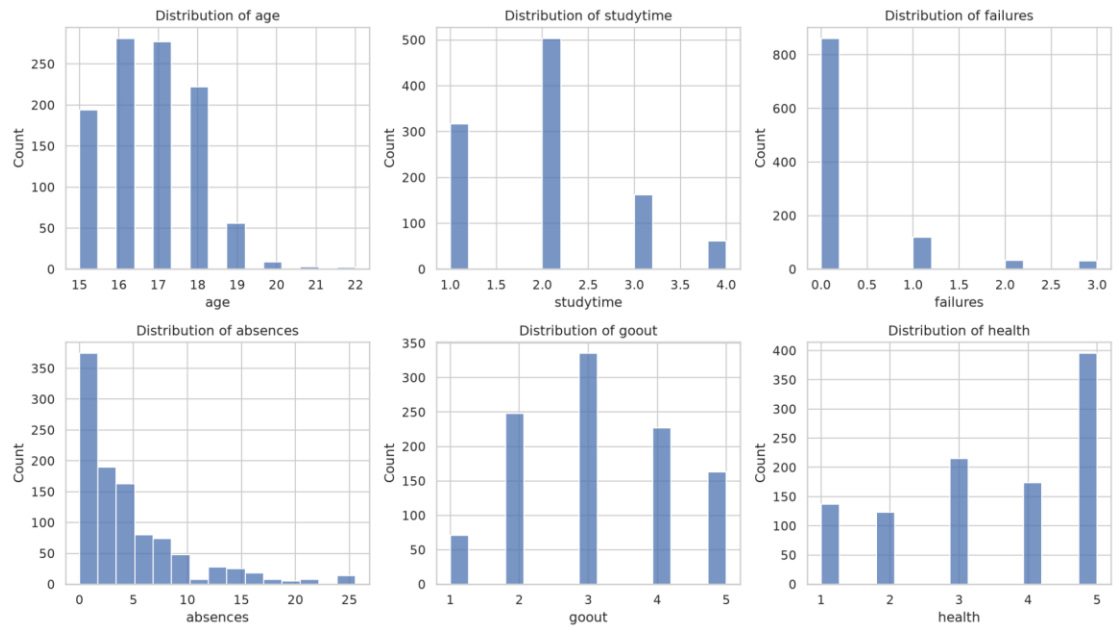


Figure 5.2: Distribution of Numeric Features

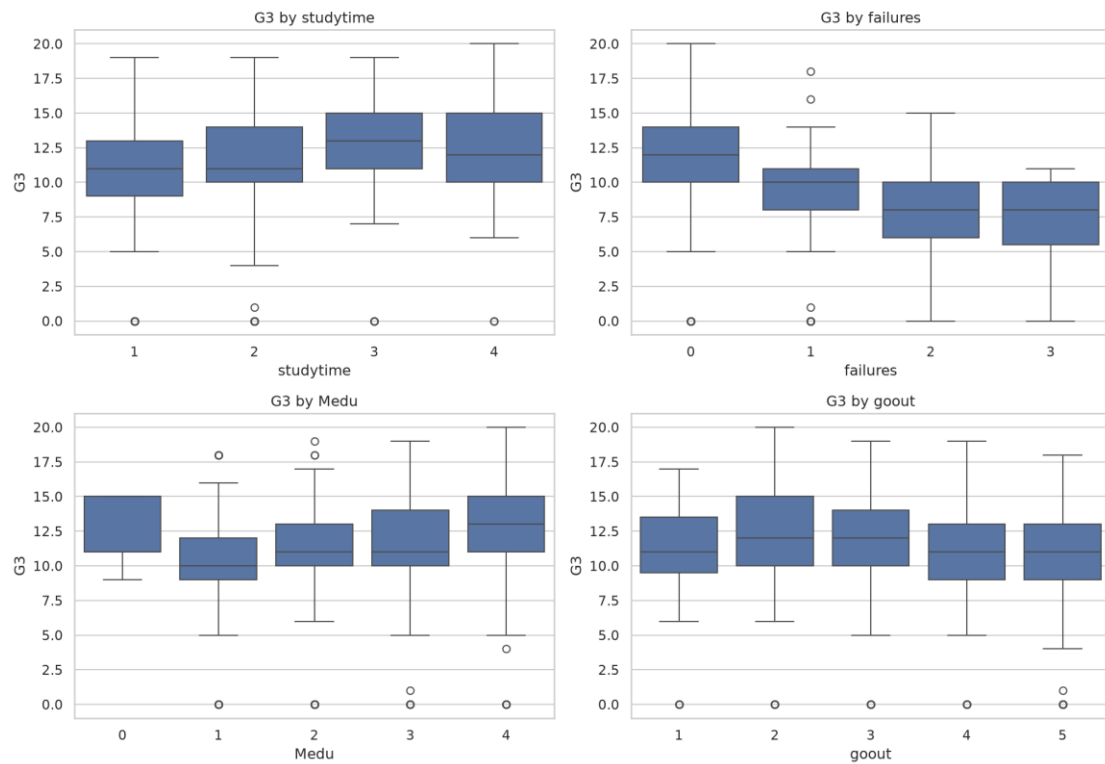


Figure 5.3: Boxplots of Grade Factors

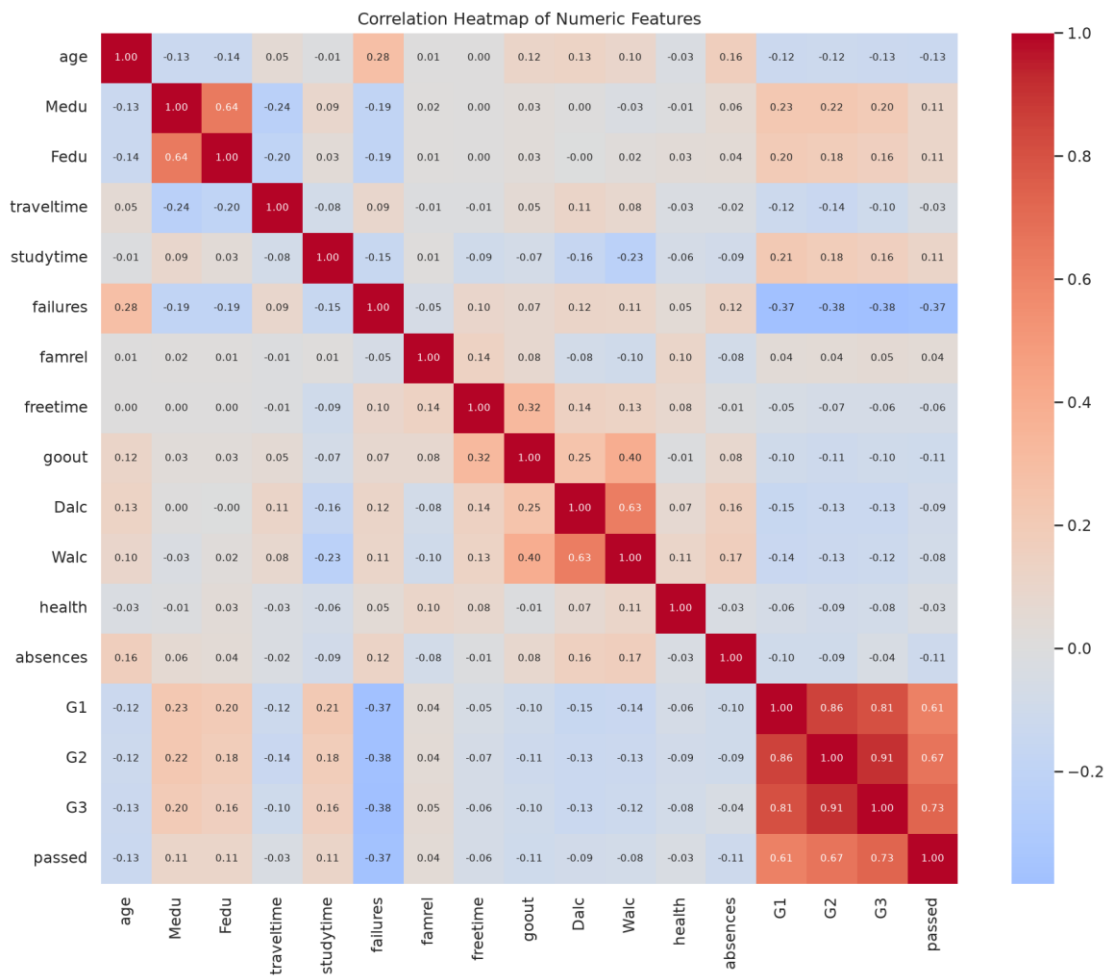


Figure 5.4: Correlation Heatmap

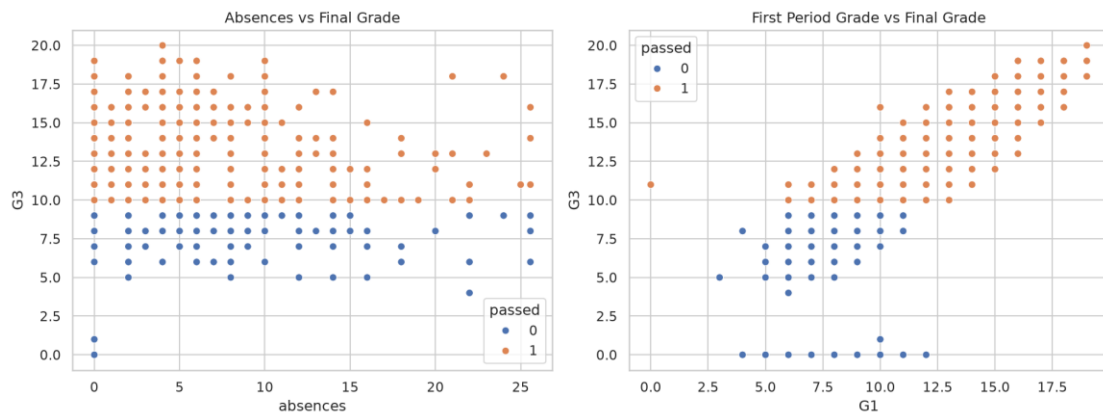


Figure 5.5: Scatter Plots

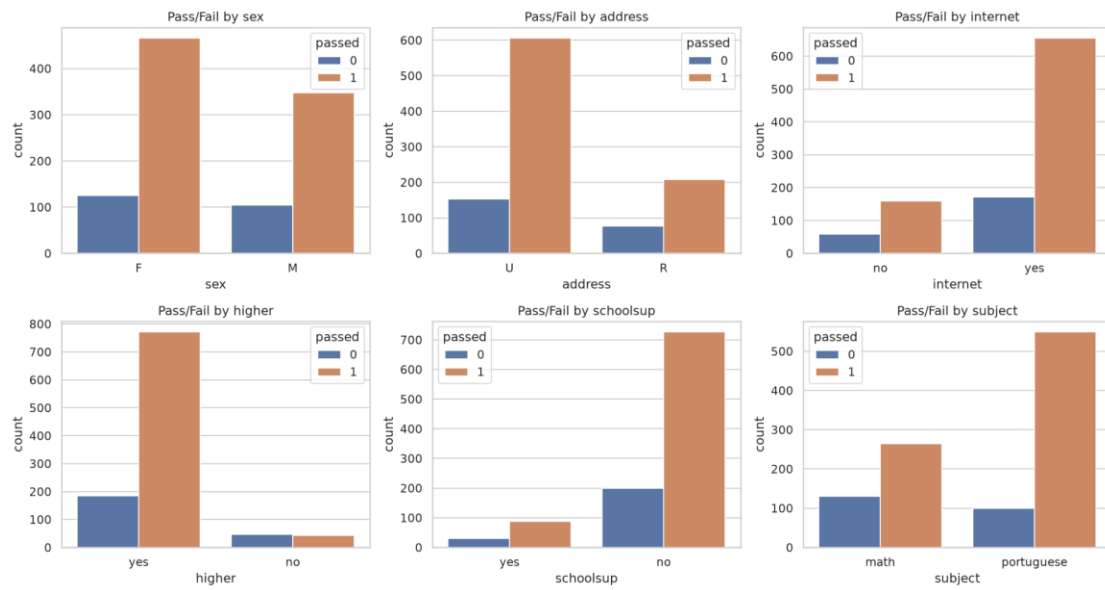


Figure 5.6: Count Plots

### Pair Plot of Key Features

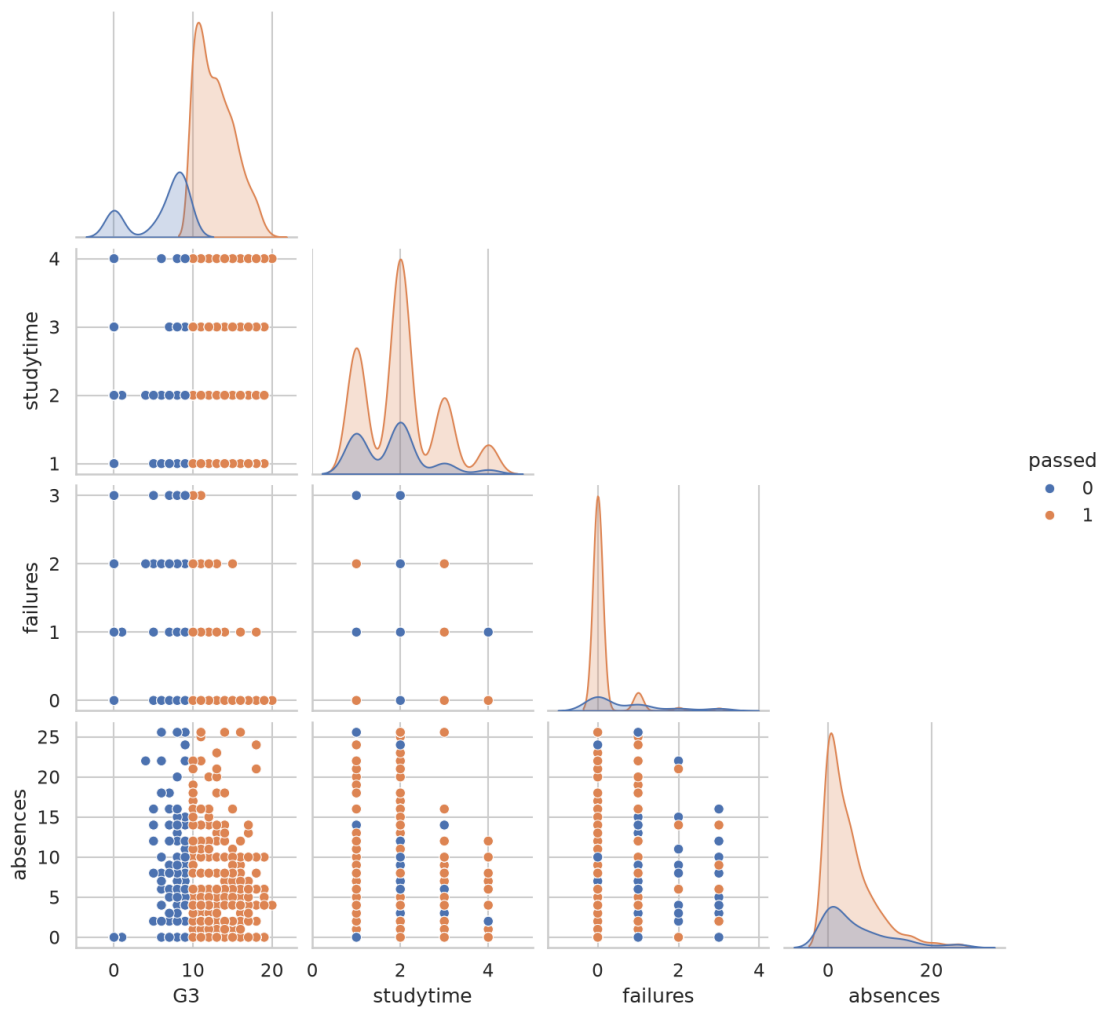


Figure 5.7: Pair Plot

A few things stood out:



The distribution of G3 is roughly bell-shaped, centered around 10–12, with a small but noticeable spike at 0 — students who essentially abandoned the course.

failures has by far the strongest negative correlation with G3. Students who've already failed a class before tend not to bounce back without some form of intervention.

Higher parental education, more weekly study time, and stated intent to pursue higher education were all positively associated with passing.

Frequent going out and higher alcohol consumption both lined up with lower grades.

As expected, G1 and G2 correlate with G3 at  $r > 0.80$  — strong confirmation that they had to be dropped to keep this an honest early-prediction problem.

Portuguese-course students passed at a noticeably higher rate than Math students, which is why we kept subject as a feature rather than discarding it.

## Feature Engineering

Rather than feeding the raw attributes straight into the models, we engineered six additional features that we thought would carry more signal than the individual raw columns on their own:

| Feature            | Definition                                        | Rationale                                                |
|--------------------|---------------------------------------------------|----------------------------------------------------------|
| parent_edu_avg     | $(\text{Medu} + \text{Fedu}) / 2$                 | Captures overall household education level in one number |
| alcohol_index      | $(5 \cdot \text{Dalc} + 2 \cdot \text{Walc}) / 7$ | A weighted measure of weekly alcohol consumption         |
| support_count      | $\text{schoolsup} + \text{famsup} + \text{paid}$  | Total number of support systems a student has access to  |
| study_social_ratio | $\text{studytime} / (\text{goout} + 1)$           | Balance between study discipline and social activity     |
| age_group          | Binned age: 15–16, 17–18, 19+                     | Allows for non-linear age effects                        |
| log_absences       | $\log(1 + \text{absences})$                       | Tames the heavy right-skew in raw absence counts         |

After encoding and feature engineering, the final feature matrix had 46 columns, which then went through a feature selection step to trim things down to the most useful predictors.

## Feature Selection

We didn't want to rely on just one selection method, so we ran four approaches spanning all three families (filter, wrapper, and embedded):

Filter — correlation analysis: top 15 features ranked by absolute point-biserial correlation with the target.

Filter — Chi-Square test: top 15 features by  $\chi^2$  score, using min-max scaled inputs.

Wrapper — Recursive Feature Elimination (RFE) with logistic regression, selecting 15 features.

Embedded — Random Forest importance: top 15 features by Gini importance, using 300 trees.

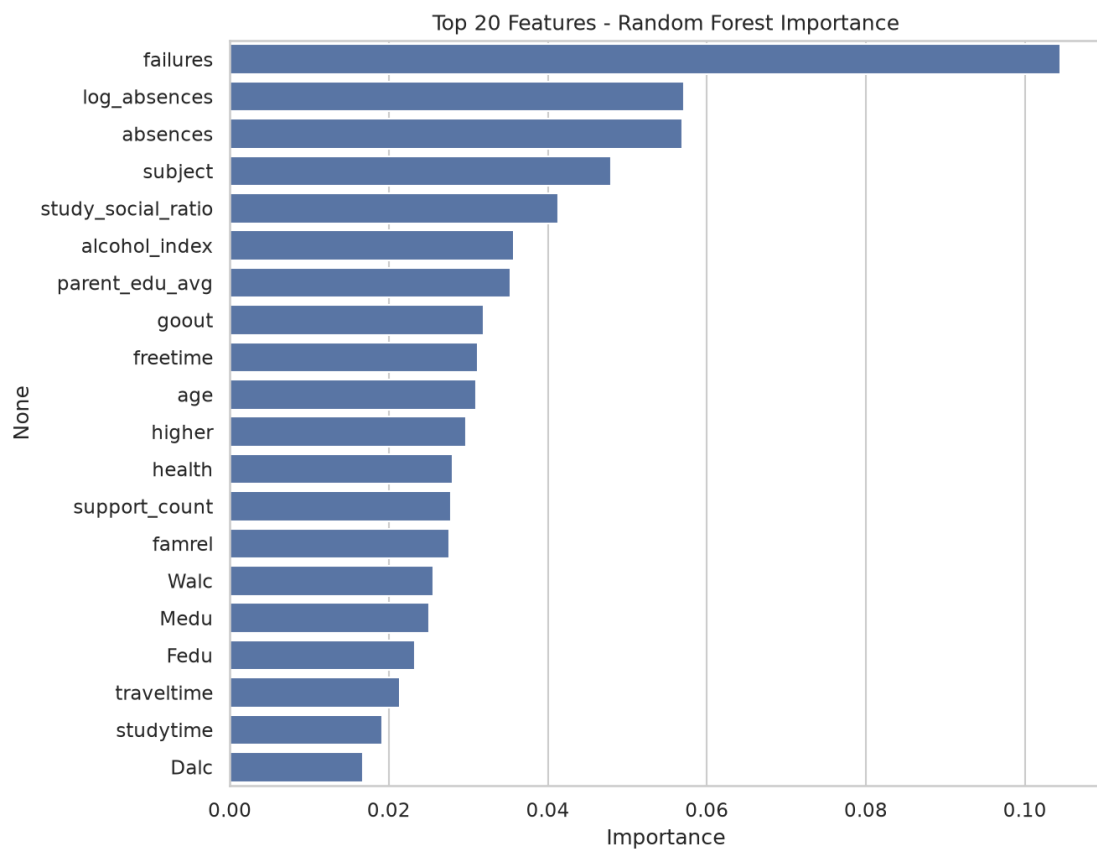


Figure 7.1: Random Forest Feature Importance

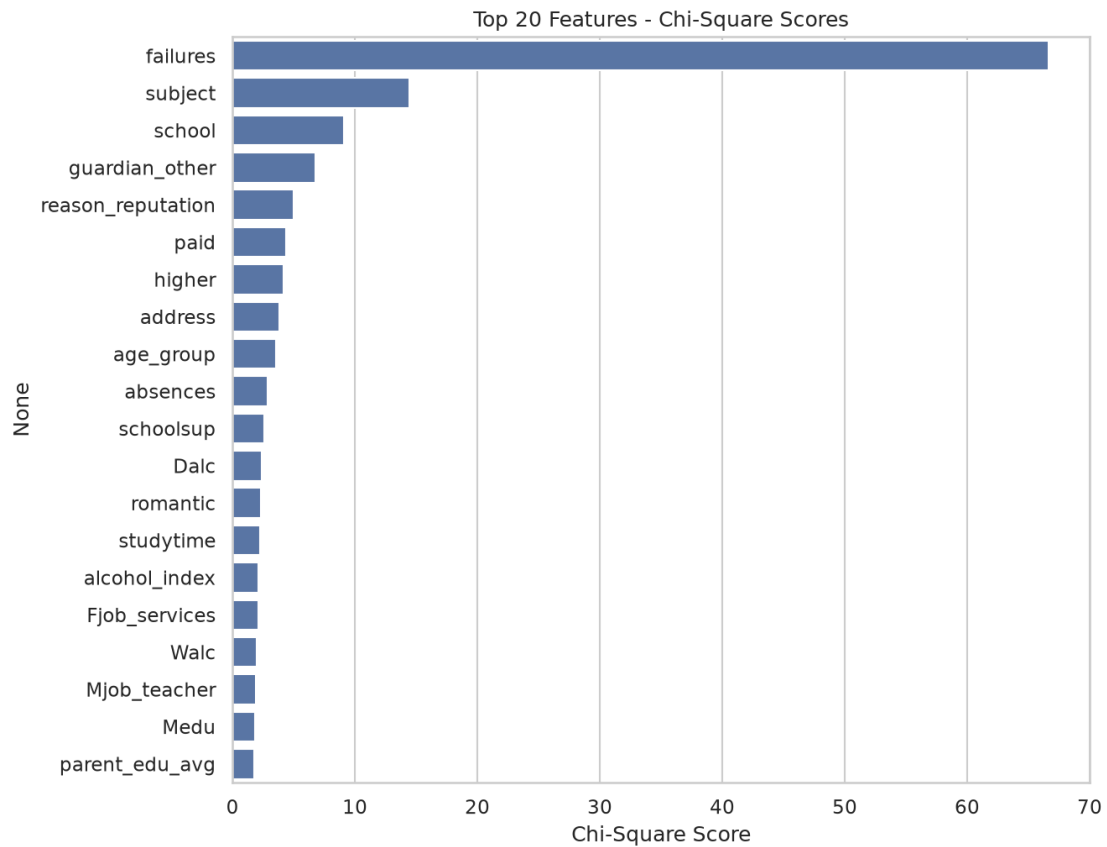


Figure 7.2: Chi-Square Feature Scores

We then kept any feature that was selected by at least two of the four methods — a simple majority-vote scheme. That left us with 16 final features: failures, higher, absences, log\_absences, subject, age, age\_group, school, goout, parent\_edu\_avg, alcohol\_index, support\_count, studytime, study\_social\_ratio, schoolsup, and guardian\_other.

This voting approach felt like a reasonable compromise — it blends univariate signal from the filter methods with the multivariate interactions RFE picks up and the non-linear effects Random Forest is good at capturing. One nice validation: 4 of our 6 engineered features made it through this selection process, which suggests the feature engineering step actually added value rather than just noise.

## Model Development

We started with Logistic Regression (`max_iter = 2000`) as a simple, interpretable baseline, then trained three more advanced models — Random Forest (300 trees), Gradient Boosting, and an SVM with an RBF kernel.

For hyperparameter tuning, we ran `GridSearchCV` (5-fold cross-validation, F1 scoring) on Random Forest, sweeping across 36 combinations of `n_estimators` {200, 400}, `max_depth` {None, 8, 14}, `min_samples_split` {2, 5, 10}, and `max_features` {sqrt, 0.5}. The search settled on `max_depth=8`, `max_features='sqrt'`, `min_samples_split=10`, `n_estimators=400`, with a best cross-validated F1 of 0.8894. On the held-out test set, the tuned model's ROC AUC improved from 0.7236 to 0.7331 — a modest but real gain in generalization from the shallower, more regularized trees — while F1 stayed essentially flat (0.8743 → 0.8725), which isn't surprising since tuning was optimizing for AUC-style ranking behaviour, not raw F1.

As a bonus experiment, we also tried PCA, reducing the 16 selected features down to 12 principal components while retaining 95% of the variance. Running Random Forest on these PCA features actually improved things slightly — accuracy went from 0.7895 to 0.8038 and F1 from 0.8743 to 0.8839 — a small but genuine improvement, and with a 25% cut in dimensionality to boot.

## Results and Discussion

Here's how all the models stacked up on the 209 held-out test students:

| Model                          | Accuracy | Precision | Recall | F1-Score | ROC AUC |
|--------------------------------|----------|-----------|--------|----------|---------|
| Logistic Regression (baseline) | 0.7943   | 0.8125    | 0.9571 | 0.8789   | 0.7147  |
| Random Forest                  | 0.7895   | 0.8182    | 0.9387 | 0.8743   | 0.7236  |
| Gradient Boosting              | 0.7943   | 0.8226    | 0.9387 | 0.8768   | 0.7507  |
| SVM (RBF kernel)               | 0.7943   | 0.8158    | 0.9509 | 0.8782   | 0.7446  |
| Random Forest (tuned)          | 0.7847   | 0.8105    | 0.9448 | 0.8725   | 0.7331  |
| Random Forest + PCA (bonus)    | 0.8038   | —         | —      | 0.8839   | —       |

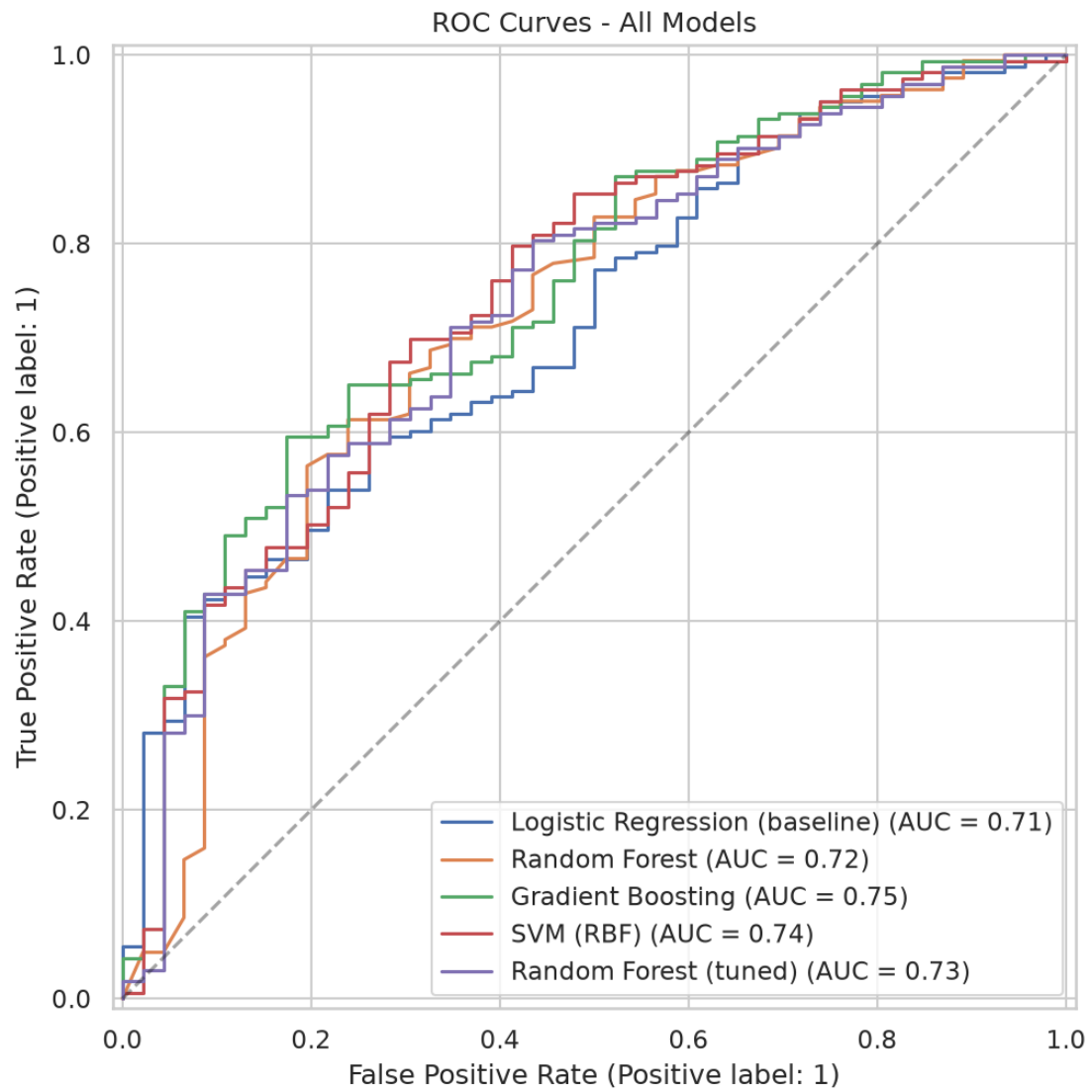


Figure 9.1: ROC Curves

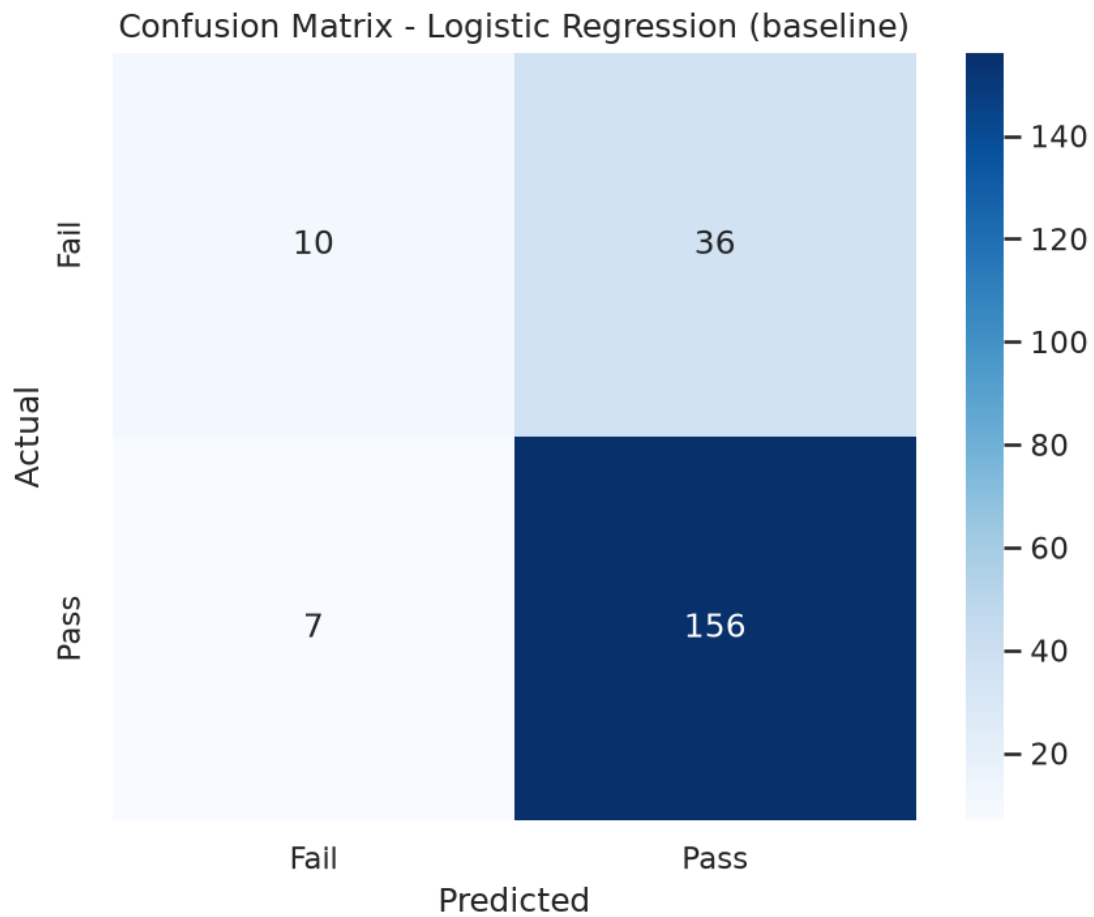


Figure 9.2: Confusion Matrix

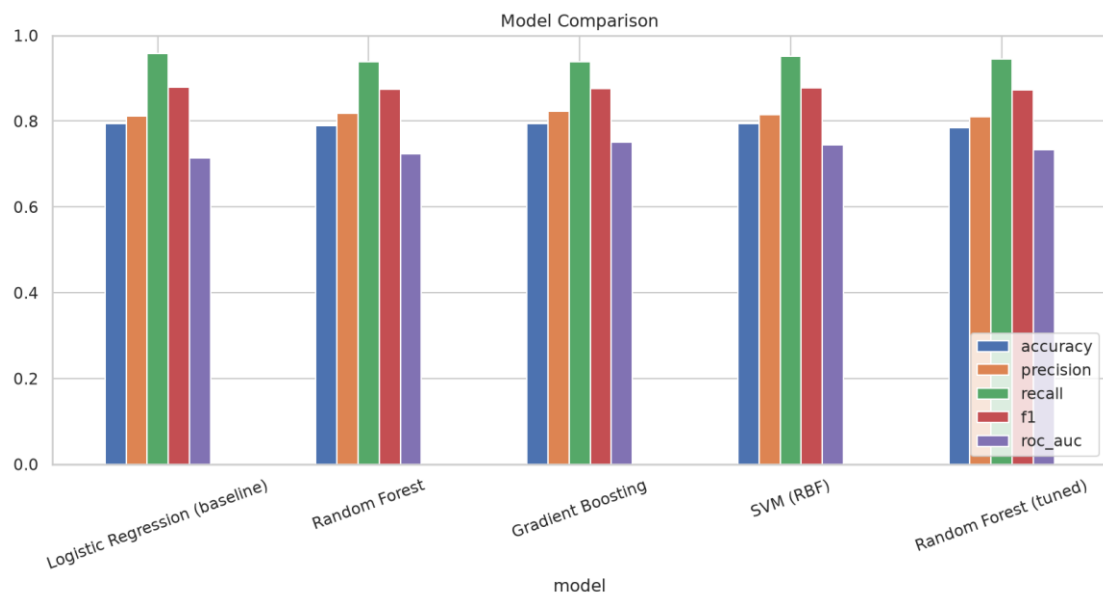


Figure 9.3: Model Comparison

A few observations worth calling out. First, every model lands in roughly the same 79–80% accuracy range — which is actually a solid result once you remember that grade history was intentionally excluded. If G1 and G2 were included, accuracy would jump past 90%, but that version of the model

would be practically useless for early intervention, since it would only 'predict' a grade the school already has. We made that trade-off on purpose.

Second, Gradient Boosting stands out with the highest ROC AUC (0.7507), meaning it's the best of the group at ranking students by risk level — which is exactly what a school would want if they're planning to target, say, the riskiest 20% of students for tutoring rather than making a hard pass/fail call for everyone.

Third, the logistic regression baseline's high F1-score is a bit deceptive — it's being pulled up by very high recall on the majority (pass) class. Looking at the confusion matrices, every single model struggles more with the fail class, which lines up with the 78/22 class imbalance we noted earlier.

Finally, tuning the Random Forest didn't dramatically change performance, but it did produce a shallower, more regularized model with better AUC and less overfitting — a reasonable trade to make even without a huge headline number.

## Conclusions and Recommendations

Overall, we think this project shows that a reasonably reliable early-warning classifier can be built using only the information available at enrollment — no exam scores required. Past failures, ambition toward higher education, absences, parental education, and social/lifestyle habits all came up repeatedly as the strongest and most consistent predictors across every selection method we tried.

A few practical recommendations follow from this:

Run the model at the very start of each academic year and flag the highest-risk group (say, the top quintile) for tutoring and counselling.

Prioritize students with prior class failures and low stated interest in higher education — these were consistently the two strongest risk signals.

Treat absences as a live, in-year signal — since it was a top predictor, ongoing attendance monitoring could catch newly at-risk students mid-year, not just at the start.

Engage parents more directly in households with lower parental education, since this factor showed up as meaningfully protective when higher.

Pair the model's output with human judgement rather than automating decisions outright — a false alarm just costs a tutoring slot, but a missed at-risk student could mean a dropout.

As for limitations, the dataset comes from two schools in Portugal back in 2008, so retraining on local, more recent data would be necessary before deploying anything like this in a different context. The class imbalance also limits how well any model can catch the fail class, and techniques like SMOTE or class-weighting would be worth exploring in future work. Finally, richer in-year signals — quiz scores, assignment completion, attendance streaks — would likely push the performance ceiling higher than what's achievable with enrollment-time data alone.

## References

Cortez, P., & Silva, A. (2008). Using Data Mining to Predict Secondary School Student Performance. Proceedings of the 5th Future Business Technology Conference (FUBUTEC 2008), pp. 5–12.

UCI Machine Learning Repository: Student Performance Data Set.  
<https://archive.ics.uci.edu/dataset/320/student+performance>

Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Waskom, M. (2021). seaborn: statistical data visualization. Journal of Open Source Software, 6(60), 3021.

Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90–95.

## GitHub Repository

The project was developed collaboratively using GitHub. A shared repository was created for the project, and all team members contributed through GitHub collaboration. The repository includes the complete source code, Jupyter Notebook, Python script, datasets, presentation, report, and project documentation.

**Repository Link:** <https://github.com/Yaseenkhan334612/Student-Performance-Analysis>

### **Project Team:**

Amin-ul-Haq  
Hoorain Eman  
Yaseen Khan